

# 计算机系统能力实训第二周周记

实训题目：Windows 命令行解释器设计

本周主题：项目框架搭建、基础命令实现与功能测试

## 1 本周实训任务

第二周的主要任务是在第一周需求分析和总体设计的基础上，进入 WinShellX 命令行解释器的编码实现阶段。本周重点完成项目基础框架、命令输入循环、命令解析和命令注册机制，并按照任务书要求实现 `cd`、`dir`、`history`、`exit`、`tasklist` 和 `taskkill` 等内部命令。

除功能实现外，本周还需要对代码结构进行整理，避免将所有逻辑集中在单个源文件中。项目开发遵循高内聚、低耦合原则，将 Shell 主控、命令解析、命令注册、内部命令和工具函数划分为不同模块，并通过编译和手工测试验证主要功能。

## 2 本周完成的主要工作

### 2.1 搭建 WinShellX 基础框架

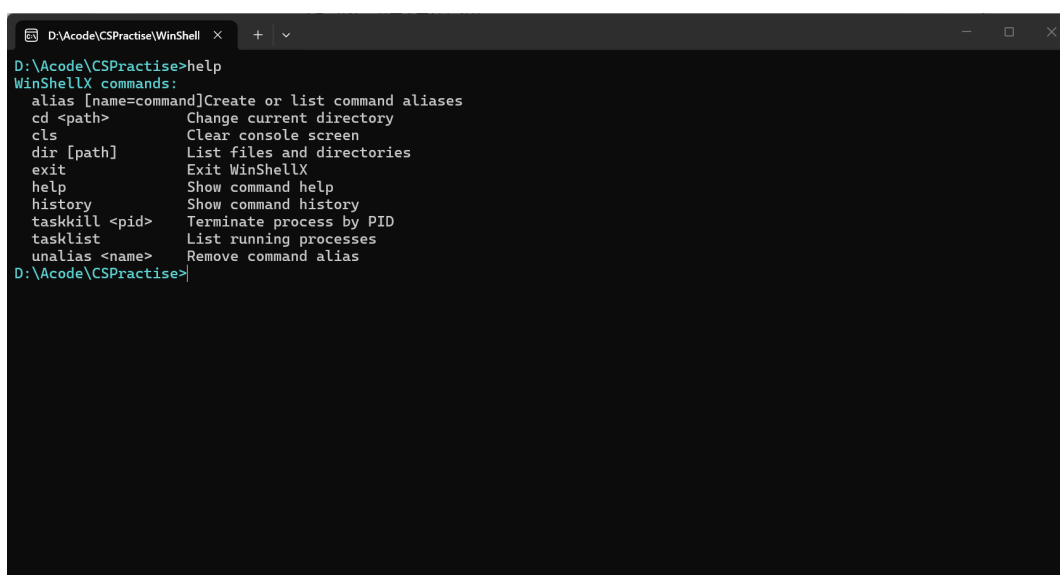
本周首先实现了 WinShellX 的程序入口和 Shell 主循环。程序启动后显示欢迎信息，然后循环获取当前工作目录，并按照“当前目录加提示符”的格式等待用户输入。例如：

```
D:\Acode\CSPractise>
```

用户输入命令后，程序去除首尾多余空格，将有效命令加入历史记录，再交给命令解析模块处理。若输入为空，则直接进入下一轮循环；若输入 `exit`，则结束主循环并退出程序。

Shell 主循环实现了命令解释器最基本的“读取、解析、查找、执行”流程，为后续增加新命令提供了统一入口。

程序启动并执行 `help` 后的界面如图 1 所示。



```
D:\Acode\CSPractise\WinShell X + v
D:\Acode\CSPractise>help
WinShellX commands:
alias [name=command] Create or list command aliases
cd <path>             Change current directory
cls                   Clear console screen
dir [path]            List files and directories
exit                 Exit WinShellX
help                 Show command help
history              Show command history
taskkill <pid>        Terminate process by PID
tasklist             List running processes
unalias <name>        Remove command alias
D:\Acode\CSPractise>
```

图 1: WinShellX 启动与帮助命令界面

2.2 实现命令解析与注册机制

为了降低主循环与具体命令之间的耦合，本周将命令解析和命令执行分离。  
命令解析模块负责把一行输入拆分为命令名和参数列表。例如：

```
cd "D:\Program Files"
```

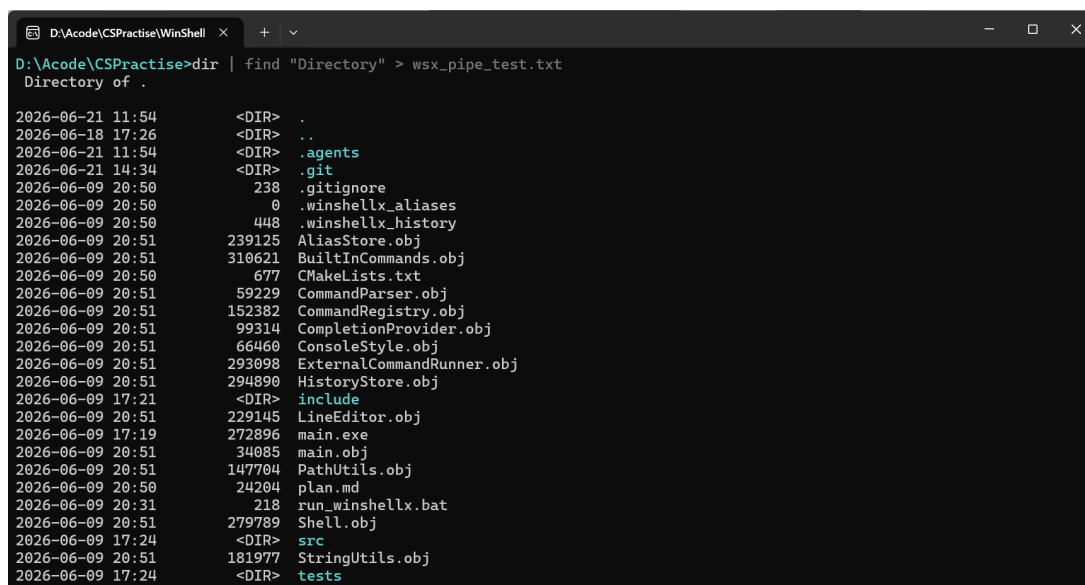
解析后得到命令名 `cd` 和路径参数 `D:\Program Files`。解析过程支持双引号包围的路径，从而避免路径中空格被错误地拆分为多个参数。命令名会统一转换为小写，使用户输入的命令不受大小写影响。  
命令注册模块使用命令名称与处理函数之间的映射保存所有内部命令。Shell 主循环只需要根据解析得到的命令名查询注册表，再调用对应处理函数。该方式使新增命令时不需要修改主循环的主体逻辑，增强了程序的可维护性和可扩展性。

2.3 实现目录与文件相关命令

本周实现了 `cd` 和 `dir` 命令。  
`cd` 命令使用 `GetCurrentDirectory` 获取当前工作目录，使用 `SetCurrentDirectory` 切换目录。当用户不提供参数时，程序输出当前目录；当用户提供路径时，程序尝试切换到指定目录。对于不存在的路径或无法访问的目录，程序读取 Windows API 错误信息并给出提示。  
`dir` 命令使用 `FindFirstFile` 和 `FindNextFile` 遍历当前目录或指定目录。输出内容包括文件修改时间、文件大小、文件名和目录标记，并统计文件数量、目录数量、文件总大小和磁盘剩余空间。完成遍历后调用 `FindClose` 关闭查找句柄，避免系统资源泄漏。

| 测试命令                         | 预期效果                   |
|------------------------------|------------------------|
| <code>cd</code>              | 显示当前工作目录。              |
| <code>cd ..</code>           | 切换到上一级目录，下一次提示符显示新的目录。 |
| <code>cd X:\not_exist</code> | 输出路径不存在或访问失败的错误信息。     |
| <code>dir</code>             | 显示当前目录中的文件和子目录。        |
| <code>dir src</code>         | 显示指定目录中的内容。            |

`dir` 命令的实际运行效果如图 2 所示。



```
D:\Acode\CSPractise>dir | find "Directory" > wxs_pipe_test.txt
Directory of .

2026-06-21 11:54      <DIR> .
2026-06-18 17:26      <DIR> ..
2026-06-21 11:54      <DIR> .agents
2026-06-21 14:34      <DIR> .git
2026-06-09 20:50        238 .gitignore
2026-06-09 20:50         0 .winshellx_aliases
2026-06-09 20:50        448 .winshellx_history
2026-06-09 20:51     239125 AliasStore.obj
2026-06-09 20:51     310621 BuiltInCommands.obj
2026-06-09 20:50         677 CMakeLists.txt
2026-06-09 20:51     59229 CommandParser.obj
2026-06-09 20:51    152382 CommandRegistry.obj
2026-06-09 20:51     99314 CompletionProvider.obj
2026-06-09 20:51    664660 ConsoleStyle.obj
2026-06-09 20:51    293098 ExternalCommandRunner.obj
2026-06-09 20:51    294890 HistoryStore.obj
2026-06-09 17:21      <DIR> include
2026-06-09 20:51    229145 LineEditor.obj
2026-06-09 17:19    272896 main.exe
2026-06-09 20:51    34085  main.obj
2026-06-09 20:51   147704 PathUtils.obj
2026-06-09 20:50    24204  plan.md
2026-06-09 20:31         218 run_winshellx.bat
2026-06-09 20:51    279789 Shell.obj
2026-06-09 17:24      <DIR> src
2026-06-09 20:51   181977 StringUtils.obj
2026-06-09 17:24      <DIR> tests
```

图 2: 目录查看命令运行效果

## 2.4 实现历史记录与退出命令

history 命令用于显示本次运行过程中输入过的命令。程序使用字符串容器按照输入顺序保存历史记录,并在执行 history 时输出命令编号和命令内容。

例如,依次输入 help、dir 和 history 后,可以看到类似结果:

```
1 help
2 dir
3 history
```

exit 命令通过修改 Shell 运行状态结束主循环,使程序能够正常退出。相比直接强制结束进程,这种方式可以为后续保存历史记录、释放资源等操作保留处理机会。

## 2.5 实现进程管理命令

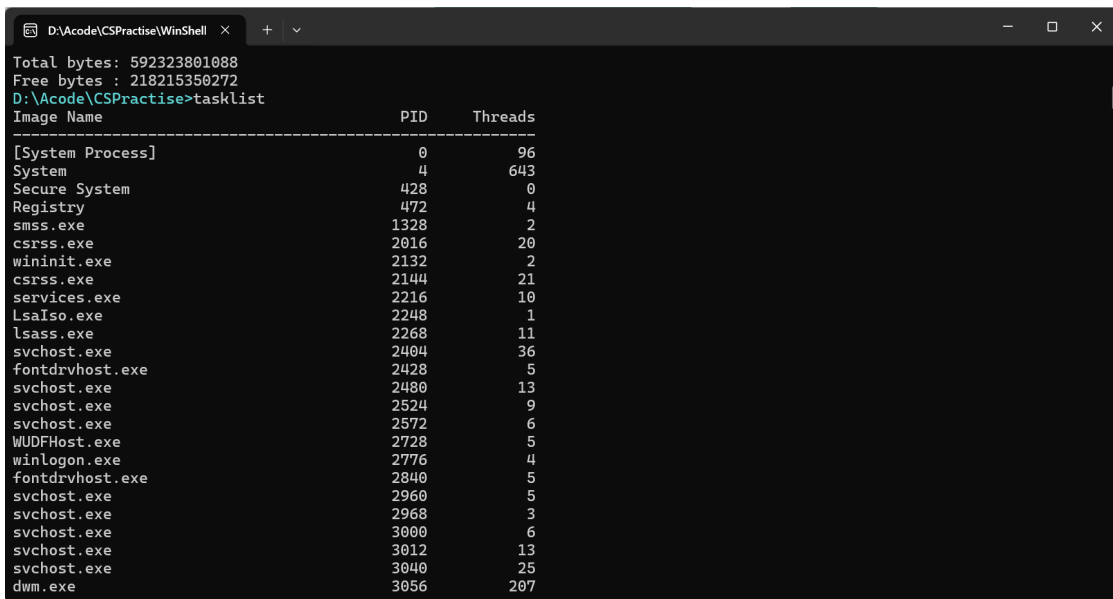
按照任务书要求,本周还实现了 tasklist 和 taskkill 命令。

tasklist 使用 CreateToolhelp32Snapshot 创建系统进程快照,然后通过 Process32First 和 Process32Next 遍历进程。程序输出进程名称、进程标识符 PID 和线程数量,使用户能够查看当前系统中的主要进程信息。

taskkill 接收一个 PID 参数。程序首先检查参数是否为有效数字,然后调用 OpenProcess 获取目标进程句柄,最后调用 TerminateProcess 结束进程。操作完成后调用 CloseHandle 关闭句柄。对于 PID 不存在、参数格式错误或权限不足等情况,程序会输出相应错误提示。

| 测试命令               | 预期效果                |
|--------------------|---------------------|
| tasklist           | 显示进程名、PID 和线程数量。    |
| taskkill           | 提示缺少 PID 参数并显示命令用法。 |
| taskkill abc       | 提示 PID 必须为数字。       |
| taskkill 999999999 | 提示无法打开不存在的进程。       |

`tasklist` 命令显示系统进程信息的效果如图 3 所示。



```

Total bytes: 592323801088
Free bytes : 218215350272
D:\Acode\CSPractise>tasklist
Image Name                                PID      Threads
-----
[System Process]                          0         96
System                                    4        643
Secure System                             428         0
Registry                                  472         4
smss.exe                                  1328         2
csrss.exe                                 2016        20
wininit.exe                               2132         2
csrss.exe                                 2144        21
services.exe                             2216         10
lsass.exe                                 2248         1
lsass.exe                                 2268        11
svchost.exe                              2404        36
fontdrvhost.exe                          2428         5
svchost.exe                              2480        13
svchost.exe                              2524         9
svchost.exe                              2572         6
WUDFHost.exe                             2728         5
winlogon.exe                             2776         4
fontdrvhost.exe                          2840         5
svchost.exe                              2960         5
svchost.exe                              2968         3
svchost.exe                              3000         6
svchost.exe                              3012        13
svchost.exe                              3040        25
dwm.exe                                  3056       207
  
```

图 3: 系统进程信息查看效果

## 2.6 整理项目目录结构

在基础功能可以运行后，本周将最初的单文件程序拆分为规范的项目结构。项目主要目录如下：

```

WinShellX/
|-- include/
|   |-- shell/
|   |-- commands/
|   `-- utils/
|-- src/
|   |-- shell/
|   |-- commands/
|   `-- utils/
|-- tests/
|-- CMakeLists.txt
`-- plan.md
  
```

`include/` 用于保存头文件, `src/shell/` 保存 Shell 主控、命令解析和命令注册代码, `src/commands/` 保存内部命令实现, `src/utils/` 保存字符串、路径和 WinAPI 错误处理等通用功能, `tests/` 保存测试用例。

同时编写了 `CMakeLists.txt`, 统一管理项目源文件和头文件路径。项目既可以使用 CMake 构建, 也可以继续使用 Visual Studio 的 `cl` 编译器直接编译。

整理后的项目目录结构如图 4 所示。

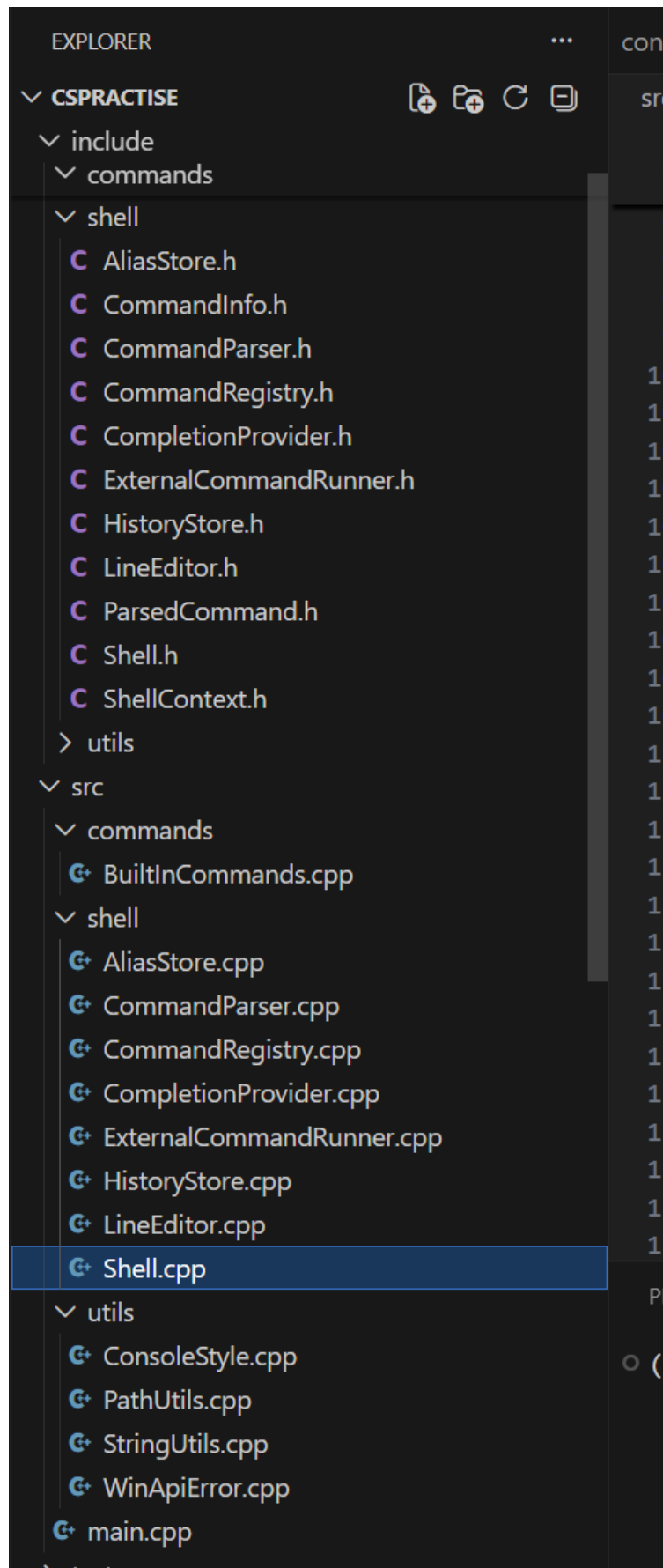


图 4: WinShellX 模块化项目目录结构

## 3 本周测试情况

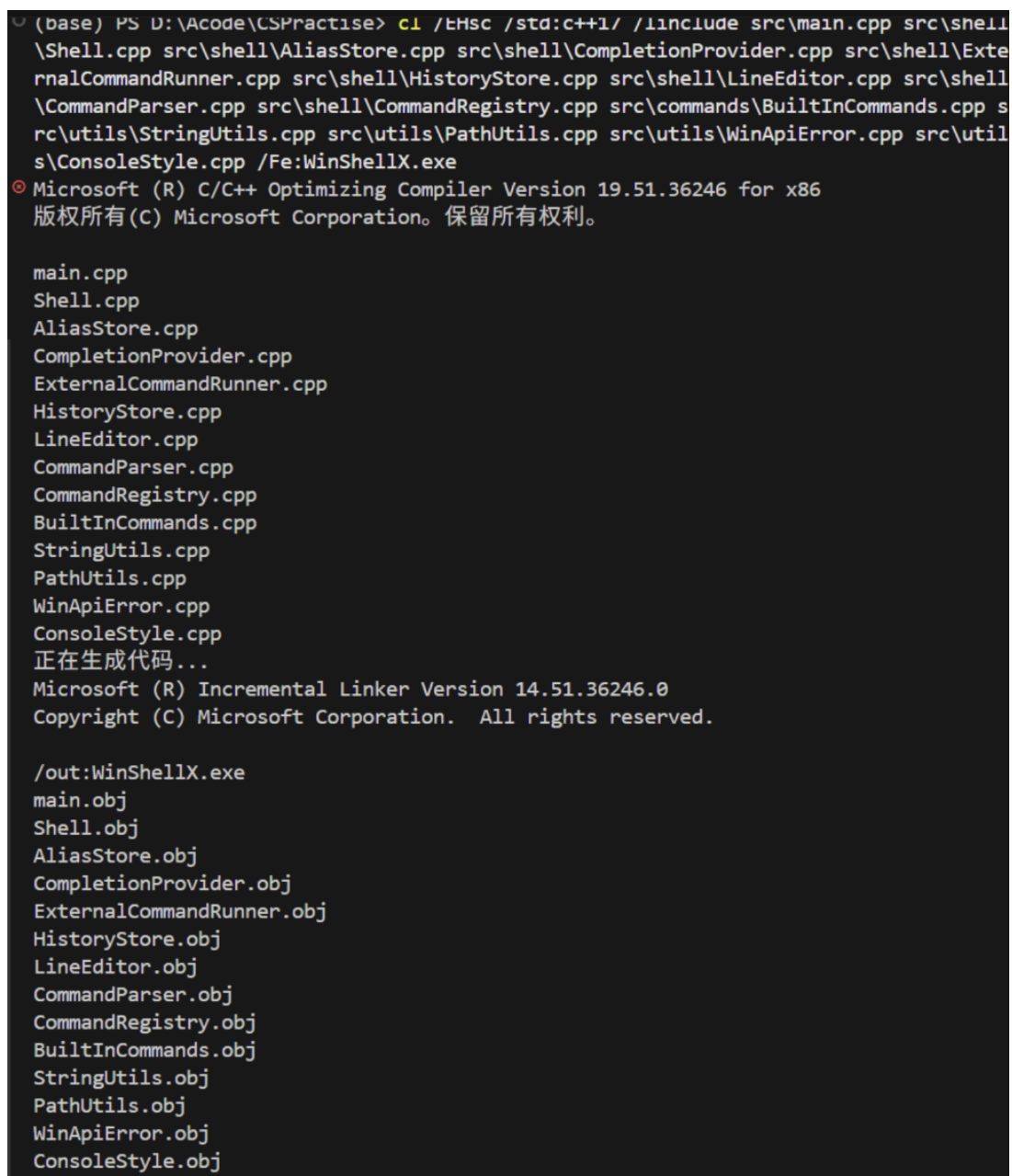
### 3.1 编译测试

完成代码拆分后，使用 C++17 标准重新编译所有源文件。编译命令中加入 /EHsc 以启用标准 C++ 异常处理语义，并使用 /Iinclude 指定头文件目录。

```
cl /EHsc /std:c++17 /Iinclude ...
```

编译过程中曾遇到进程结构体名称与 SDK 定义不一致的问题。最初使用的 PROCESSENTRY32A 在当前开发环境中无法识别，改用通用的 PROCESSENTRY32、Process32First 和 Process32Next 后编译成功。

项目使用 MSVC 编译成功的结果如图 5 所示。



```
(base) PS D:\Acode\CSPractise> cl /EHsc /std:c++17 /Iinclude src\main.cpp src\shell
\Shell.cpp src\shell\AliasStore.cpp src\shell\CompletionProvider.cpp src\shell\Exte
rnalCommandRunner.cpp src\shell\HistoryStore.cpp src\shell\LineEditor.cpp src\shell
\CommandParser.cpp src\shell\CommandRegistry.cpp src\commands\BuiltInCommands.cpp s
rc\utils\StringUtils.cpp src\utils\PathUtils.cpp src\utils\WinApiError.cpp src\util
s\ConsoleStyle.cpp /Fe:WinShellX.exe
Microsoft (R) C/C++ Optimizing Compiler Version 19.51.36246 for x86
版权所有(C) Microsoft Corporation。保留所有权利。

main.cpp
Shell.cpp
AliasStore.cpp
CompletionProvider.cpp
ExternalCommandRunner.cpp
HistoryStore.cpp
LineEditor.cpp
CommandParser.cpp
CommandRegistry.cpp
BuiltInCommands.cpp
StringUtils.cpp
PathUtils.cpp
WinApiError.cpp
ConsoleStyle.cpp
正在生成代码...
Microsoft (R) Incremental Linker Version 14.51.36246.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:WinShellX.exe
main.obj
Shell.obj
AliasStore.obj
CompletionProvider.obj
ExternalCommandRunner.obj
HistoryStore.obj
LineEditor.obj
CommandParser.obj
CommandRegistry.obj
BuiltInCommands.obj
StringUtils.obj
PathUtils.obj
WinApiError.obj
ConsoleStyle.obj
```

图 5: WinShellX 项目编译成功界面

### 3.2 功能测试

本周采用手工测试方式验证内部命令。测试覆盖正常输入、缺少参数、非法路径、非法 PID 和未知命令等情况。

主要测试结果如下：

| 测试功能     | 测试结果 | 说明                     |
|----------|------|------------------------|
| 提示符显示    | 通过   | 能够显示当前目录和提示符。          |
| 命令解析     | 通过   | 能够解析普通参数和带空格的引号路径。     |
| cd       | 通过   | 能切换目录并处理非法路径。          |
| dir      | 通过   | 能显示目录内容、文件大小和磁盘空间。     |
| history  | 通过   | 能按顺序显示本次运行中的历史命令。      |
| exit     | 通过   | 能正常结束 Shell 主循环。       |
| tasklist | 通过   | 能显示进程名、PID 和线程数。       |
| taskkill | 通过   | 能校验 PID，并对权限或进程错误给出提示。 |

经过测试，本周实现的基础功能可以稳定运行，达到了任务书对常用内部命令的基本要求。

## 4 本周遇到的问题及解决办法

### 4.1 单文件代码不便于扩展

初步版本将命令解析、Windows API 调用和主循环全部写在一个 `main.cpp` 中。虽然能够快速验证功能，但代码长度增加后，各模块职责混杂，继续增加命令会使维护难度迅速上升。

为解决这一问题，本周将代码拆分为 Shell、命令、工具和测试等模块。主循环只负责命令调度，具体命令由独立处理函数完成，通用逻辑放入工具模块。拆分后，各模块职责更加清晰，也便于后续继续添加功能。

### 4.2 Windows 句柄需要正确释放

文件遍历、进程快照和进程控制都会产生 Windows 句柄。如果只完成操作而不关闭句柄，程序长时间运行后可能产生资源泄漏。

在实现过程中，我对每个获取句柄的 API 进行了对应检查：文件查找结束后调用 `FindClose`，进程快照和进程对象使用完毕后调用 `CloseHandle`。同时在失败分支中也注意关闭已经成功创建的句柄。

### 4.3 控制台输出格式相互影响

在测试 `dir` 命令时，曾出现日期显示顺序异常的问题。分析后发现，前一个命令使用的 `std::left` 输出状态会继续影响后续输出，使日期中的数字采用左对齐并被零填充。

解决方法是在输出日期前显式设置 `std::right`，并在需要时重新设置填充字符。这个问题让我认识到，C++ 流格式状态会持续保留，编写多个控制台输出模块时必须主动恢复或设置所需格式。

#### 4.4 进程终止存在权限限制

部分系统进程受到 Windows 保护，普通权限下无法通过 `OpenProcess` 获取终止权限。程序不能将所有失败都理解为代码错误，而应检查 API 返回值并输出系统错误信息。

因此，本周将 WinAPI 错误码转换封装为通用工具。当调用失败时，程序使用 `GetLastError` 和 `FormatMessage` 获取可读错误信息，帮助用户区分参数错误、进程不存在和权限不足等情况。

## 5 本周收获

### 5.1 理解了命令解释器的核心执行流程

通过实际编码，我将第一周学习的理论流程落实到了程序中。一个命令从用户输入到最终执行，需要依次经过字符串读取、命令解析、注册表查询、处理函数调用、Win32 API 执行和结果输出。模块化实现后，这一流程更加清晰。

### 5.2 提高了 Win32 API 实践能力

本周实际使用了目录、文件、进程和错误处理相关 API。相比只阅读文档，编码和调试让我更深入地理解了 API 参数、结构体初始化、返回值检查、句柄管理和权限控制等问题。

### 5.3 认识到工程结构的重要性

将单文件代码拆分为多个模块后，我直观感受到良好工程结构对项目扩展的帮助。高内聚、低耦合不仅是设计原则，也会直接影响调试效率、测试难度和后续功能扩展成本。

## 6 下周计划

### 6.1 完善历史命令功能

下周计划将历史命令从当前会话保存扩展为文件持久化，程序退出时保存历史记录，启动时重新加载。同时研究使用方向键浏览历史命令和根据输入前缀搜索历史命令。

### 6.2 增强控制台交互

计划增加 `help` 和 `cls` 命令，并研究控制台按键读取、命令提示和 `Tab` 补全。补全内容可以包括历史命令、内部命令名称和当前目录中的路径。

### 6.3 实现外部命令执行

计划研究 Windows 外部命令的查找和执行方式，包括 `PATH`、`PATHEXT`、`CreateProcess` 以及批处理文件的执行机制，使 WinShellX 除内部命令外，还可以运行 `notepad`、`code` 等外部程序。

## 6.4 继续补充测试

下周将在现有手工测试的基础上增加历史记录、智能补全和外部命令测试，并继续完善错误处理和测试文档。

## 7 本周小结

第二周完成了 WinShellX 基础框架和任务书要求的主要内部命令，实现了目录切换、目录遍历、历史显示、程序退出、进程枚举和进程终止等功能。项目也由初始单文件程序调整为模块化目录结构，并加入 CMake 构建配置和手工测试文档。

通过本周实践，我进一步理解了命令解释器从输入到执行的完整流程，也掌握了目录、文件和进程相关 Win32 API 的基本使用方法。当前版本已经具备命令行解释器的基础形态，下周将在此基础上重点增强历史记录、智能补全和外部命令执行能力。